Bank Management Database System

Project Phase 3 Report: SQL Queries & Relational Algebra

Group 10
Borhan Javadian (35012)
Bardiya Shavandi (33588)
Faraz Sahabi (34557)

March 30, 2026

1 Database Description

This project implements a comprehensive Bank Management Database System designed to model the operations of financial institutions. The database tracks the hierarchical structure of banks operating across multiple countries, their physical branches, and the workforce of managers and employees. It also manages customer data, account holdings, and a variety of financial transactions including specialized subtypes such as transfers and loans.

2 SQL Queries and Relational Algebra

2.1 Queries by Borhan Javadian (35012)

Query 1: Selection + Projection

Description: Retrieve the full name and salary of employees whose salary is greater than 50,000.

```
SELECT FullName, Salary FROM Employee WHERE Salary > 50000;
```

Relational Algebra:
$\pi_{FullName,Salary}(\sigma_{Salary>50000}(Employee))$

Query 2: Join between 3 Tables

Description: List each account number along with the owner's full name and the branch address.

```
SELECT A.Number, C.FullName, B.Address
FROM Account A, Customer C, Branch B
WHERE A.CustomerID = C.CustomerID AND A.BranchID = B.BranchID;
```

Relational Algebra:
$\pi_{Number,FullName,Address}(Account \bowtie_{CustomerID} Customer \bowtie_{BranchID} Branch)$

Query 3: Aggregation with Having

Description: Show each account ID and its total transaction amount, only where total exceeds 1,000.

```
SELECT AccountID, SUM(Amount) AS TotalAmount
FROM Transaction GROUP BY AccountID HAVING SUM(Amount) > 1000;
```

Relational Algebra:
$\sigma_{TotalAmount>1000}({}_{AccountID}\gamma_{SUM(Amount)\rightarrow TotalAmount}(Transaction))$

Query 4: Sorting Multiple Columns

Description: List all employees sorted alphabetically by position, then by salary descending.

```
SELECT FullName, Position, Salary FROM Employee
ORDER BY Position ASC, Salary DESC;
```

Relational Algebra:

$\tau_{Position\uparrow,Salary\downarrow}(\pi_{FullName,Position,Salary}(Employee))$

Query 5: Nested Subquery

Description: List employees who earn more than the average salary of all employees.

```
SELECT FullName, Salary FROM Employee
WHERE Salary > (SELECT AVG(Salary) FROM Employee);
```

Relational Algebra:

$\pi_{FullName,Salary}(\sigma_{Salary>avg\_sal}(Employee \times \gamma_{AVG(Salary)\rightarrow avg\_sal}(Employee)))$

2.2 Queries by Bardiya Shavandi (33588)

Query 6: Join + Aggregate + Sort

Description: Show each customer's total account balance, ordered from highest to lowest.

```
SELECT C.FullName, SUM(A.Balance) AS TotalBalance
FROM Customer C, Account A WHERE C.CustomerID = A.CustomerID
GROUP BY C.CustomerID, C.FullName ORDER BY TotalBalance DESC;
```

Relational Algebra:

$\tau_{TotalBalance\downarrow}(_{CustomerID,FullName}\gamma_{SUM(Balance)\rightarrow TotalBalance}(Customer \bowtie Account))$

Query 7: EXISTS Logic

Description: List customers who have made at least one transaction on any of their accounts.

```
SELECT DISTINCT C.FullName FROM Customer C
WHERE EXISTS (SELECT * FROM Account A, Transaction T
              WHERE A.AccountID = T.AccountID AND A.CustomerID = C.CustomerID);
```

Relational Algebra:

$\pi_{FullName}(Customer \bowtie (\pi_{CustomerID}(Account \bowtie Transaction)))$

Query 8: Count with Grouping

Description: Count the number of accounts at each branch, showing branch address and count.

```
SELECT B.Address, COUNT(A.AccountID) AS AccountCount
FROM Branch B, Account A WHERE B.BranchID = A.BranchID
GROUP BY B.BranchID, B.Address;
```

Relational Algebra:

$\pi_{Address,AccountCount}(_{BranchID,Address}\gamma_{COUNT(AccountID)\rightarrow AccountCount}(Branch \bowtie Account))$

Query 9: Set Difference (NOT IN)

Description: List the names of customers who have never taken a loan.

```
SELECT FullName FROM Customer WHERE CustomerID NOT IN
(SELECT A.CustomerID FROM Account A, Transaction T, Loan L
 WHERE A.AccountID = T.AccountID AND T.TransactionID = L.TransactionID);
```

Relational Algebra:
$\pi_{FullName}(Customer) \setminus \pi_{FullName}(Customer \bowtie Account \bowtie Transaction \bowtie Loan)$

Query 10: Basic Retrieval

Description: Retrieve all information about every customer in the database.

```
SELECT * FROM Customer;
```

Relational Algebra:
$Customer$

2.3 Queries by Faraz Sahabi (34557)

Query 11: Single Column Sort

Description: List all account numbers and balances ordered from lowest to highest balance.

```
SELECT Number, Balance FROM Account ORDER BY Balance ASC;
```

Relational Algebra:
$\tau_{Balance\uparrow}(\pi_{Number,Balance}(Account))$

Query 12: Complex Join

Description: List employee names and branch addresses for those earning above 60,000.

```
SELECT E.FullName, B.Address FROM Employee E, WorksIn W, Branch B
WHERE E.EmployeeID = W.EmployeeID AND W.BranchID = B.BranchID AND E.Salary > 60000;
```

Relational Algebra:
$\pi_{FullName,Address}(\sigma_{Salary>60000}(Employee) \bowtie WorksIn \bowtie Branch)$

Query 13: Average with Having

Description: Average transaction amount per currency, where average exceeds 500.

```
SELECT Currency, AVG(Amount) AS AvgAmount FROM Transaction
GROUP BY Currency HAVING AVG(Amount) > 500;
```

Relational Algebra:
$\sigma_{AvgAmount>500}({}_{Currency}\gamma_{AVG(Amount)\rightarrow AvgAmount}(Transaction))$

Query 14: Filtered Sort

Description: Show number and balance of positive accounts, sorted highest to lowest.

```
SELECT Number, Balance FROM Account WHERE Balance > 0 ORDER BY Balance DESC;
```

Relational Algebra:
$\tau_{Balance\downarrow}(\pi_{Number,Balance}(\sigma_{Balance>0}(Account)))$

Query 15: Join with Aggregation

Description: List each bank's name and its number of branches, for banks with more than one branch.

```
SELECT BK.Name, COUNT(BR.BranchID) AS BranchCount FROM Bank BK, Branch BR
WHERE BK.BankID = BR.BankID GROUP BY BK.BankID, BK.Name HAVING COUNT(BR.BranchID) > 1
```

Relational Algebra:
$\sigma_{BranchCount>1}(\pi_{Name,BranchCount}({}_{BankID,Name}\gamma_{COUNT(BranchID)\rightarrow BranchCount}(Bank \bowtie Branch)))$

3 Appendix: AI Usage Documentation

- **Claude:** `https://claude.ai/share/cf1aea08-1950-422f-b565-6b7b838cc1f1`
- **Claude:** `https://claude.ai/share/efc35273-633a-485d-b5ab-da4a0aa0cd55`
- **Gemini:** `https://gemini.google.com/share/bc2779df6e9f`